

人工智能基础实验三

实验报告

实验题目: K-means 聚类算法的实现与应用

专 业: 人工智能

姓 名: 马承乾

学 号: 22920212204180

实验日期: 2023.11.23

一. 实验题目

- K-means 聚类算法的实现与应用

二. 实验目的

- 理解 K-means 聚类算法的原理和基本步骤。
- 学会使用 Python 编程语言实现 K-means 聚类算法。
- 掌握 K-means 算法在实际数据集上的应用和可视化。

三. 实验方法

1. 实验环境准备：

在本次实验中，我们将使用 Python 编程语言及其相关库进行算法实现和数据处理。确保已经安装了以下库：

- NumPy: 用于数值计算。
- Matplotlib: 用于可视化。
- scikit-learn: 用于生成模拟数据集。

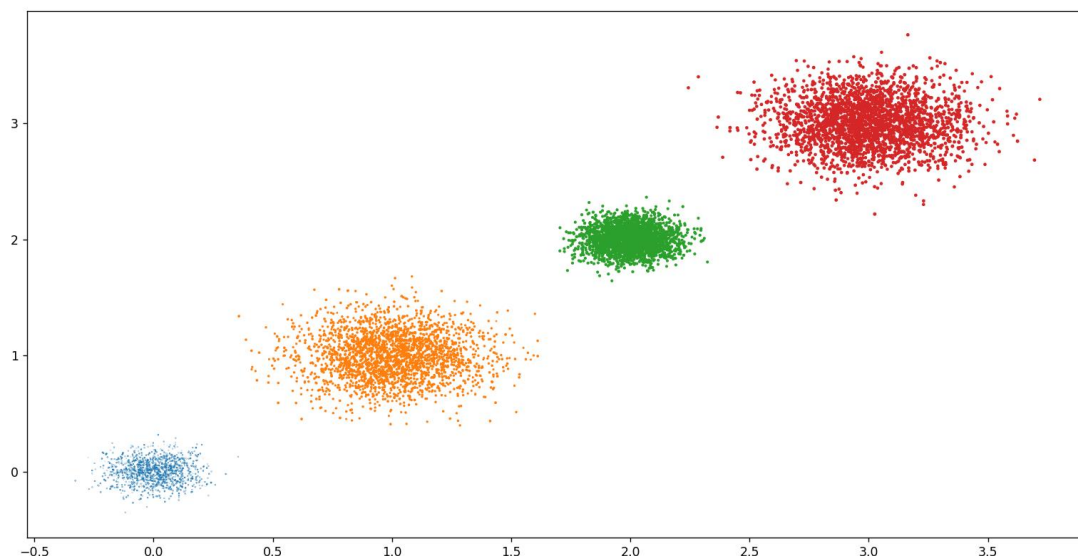
2. 实验数据准备：

选择一个模拟数据集，确保数据集的特征维度和样本数量适中。这里，我们通过 sklearn 库直接生成数据集，代码如下：

```
X, y = datasets.make_blobs(n_samples=10000,  
                           n_features=3,  
                           centers=[[0,0,0], [1,1,1], [2,2,2],  
[3,3,3]],  
                           cluster_std=[0.1, 0.2, 0.1, 0.2],  
                           random_state=5)
```

另样本数量为 10000，维数为 3，取四个簇中心，并确定随机种子，保证每次生成的数据都相同。

通过 matplotlib 进行可视化，得到初始数据分布如下：



3. K-means 算法原理:

K-means 是一种迭代的、基于划分的聚类算法，其目标是将数据集划分为 K 个不同的簇，使得簇内的数据点相似度较高，而簇间的相似度较低。算法的基本原理可以概括为以下几个步骤：

初始化聚类中心： 从数据集中随机选择 K 个样本作为初始的聚类中心。

样本归类： 对数据集中的每个样本，计算其与 K 个聚类中心的距离，将样本分配给距离最近的聚类中心所在的簇。

更新聚类中心： 对每个簇，计算其所有样本的均值，将均值作为新的聚类中心。

重复步骤 2 和 3： 重复执行样本归类和聚类中心更新的步骤，直到满足停止条件，例如达到最大迭代次数或聚类中心几乎不再变化。

输出结果： 最终得到 K 个簇，每个簇包含一组相似的数据点，形成聚类结果。

优化目标：

K-means 的优化目标是最小化所有簇内样本的平方误差和，即最小化簇内样本与其所在簇的聚类中心之间的距离。该目标函数通常称为**簇内平方和** (Within-Cluster Sum of Squares, WCSS)，可以表示为：

$$WCSS = \sum_{i=1}^K \sum_{j=1}^{n_i} \|x_{ij} - \mu_i\|^2$$

其中，K 是簇的数目， n_i 是第 i 个簇的样本数， x_{ij} 是第 i 个簇中的第 j 个样本， μ_i 是第 i 个簇的聚类中心。

K-means 算法通过迭代优化该目标函数，不断更新聚类中心，将数据点划分到不同的簇中，直至满足停止条件。

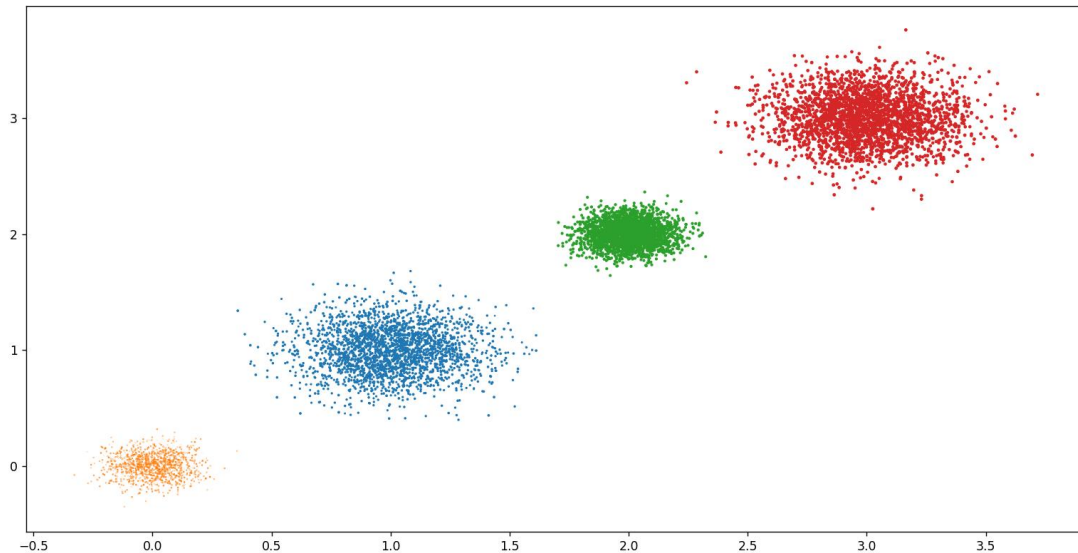
4. K-means 算法实现:

使用 Python 编写 K-means 聚类算法的代码。按照以下步骤进行实现：（具体代码见附录，代码中包含详细的注释）

- 正则化数据集。
- 计算欧氏距离的平方。
- 初始化聚类中心。
- 创建样本归类。
- 更新聚类中心。
- 遍历以上步骤直到满足收敛条件。

5. 数据加载和可视化：

使用 Matplotlib 库加载和可视化数据集。对模拟数据集生成过程进行可视化说明，以便理解数据的分布情况，在可视化结果中，一种颜色意味着预测出的一个簇，真实分布观察位置即可知，例如下面这张图，四种数据被分成了橙色、蓝色、绿色、红色四个簇。



6. 实验应用：

使用实现的 K-means 算法对选择的数据集进行聚类，并可视化展示聚类结果。解释聚类结果的含义。

在此步骤中，我们通过更改超参数来观察 K-means 的具体效果。我们能控制的参数有以下三个：

更新 K（簇的数目）：

影响聚类效果：增加或减少 K 可能会对聚类效果产生重大影响。较小的 K 可能导致簇内部的样本过于杂乱，而较大的 K 可能导致簇之间的相似性不足。

运行时间：K 的增加通常会导致计算距离和更新聚类中心的计算量增加，从而增加算法的运行时间。

调参方式：修改这两处参数即可。

```
# 聚类
clf = KMeans(k=2)
y_pred = clf.predict(X)

# 可视化
fig = plt.figure(figsize=(12, 8))
ax = Axes3D(fig, rect=[0, 0, 1, 1], elev=30, azimuth=20)
for i in range(2):
    plt.scatter(X[y_pred==i][:, 0], X[y_pred==i][:, 1], X[y_pred==i][:, 2])
    # plt.scatter(X[y==i][:, 0], X[y==i][:, 1], X[y==i][:, 2])# True
plt.show()
```

更新最大迭代次数：

影响聚类效果：增加最大迭代次数可能使得算法更有机会收敛到更好的结果，尤其是在数据集较大或者簇形状较复杂的情况下。

运行时间： 增加最大迭代次数会增加算法的运行时间。在一定程度上，可以通过观察聚类结果的变化情况来决定是否继续增加迭代次数。

修改方法：

```
# 聚类
clf = KMeans(k=4, max_iterations=500)
y_pred = clf.predict(X)
```

更新 varepsilon:

修改方法如下（默认为 $1e-4$ ）：

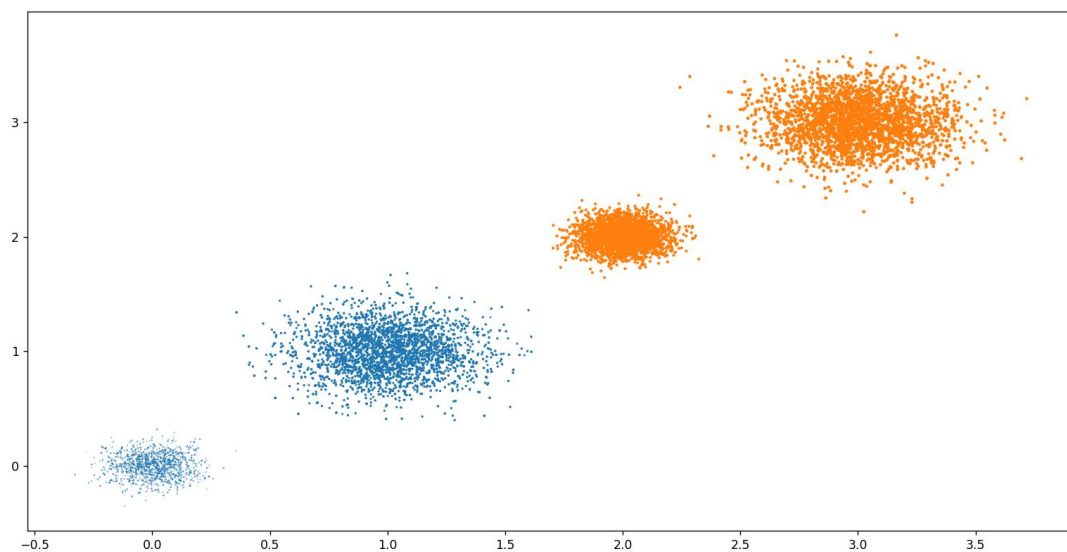
```
# 聚类
clf = KMeans(k=4, varepsilon=1e-4)
y_pred = clf.predict(X)
```

四、实验结果

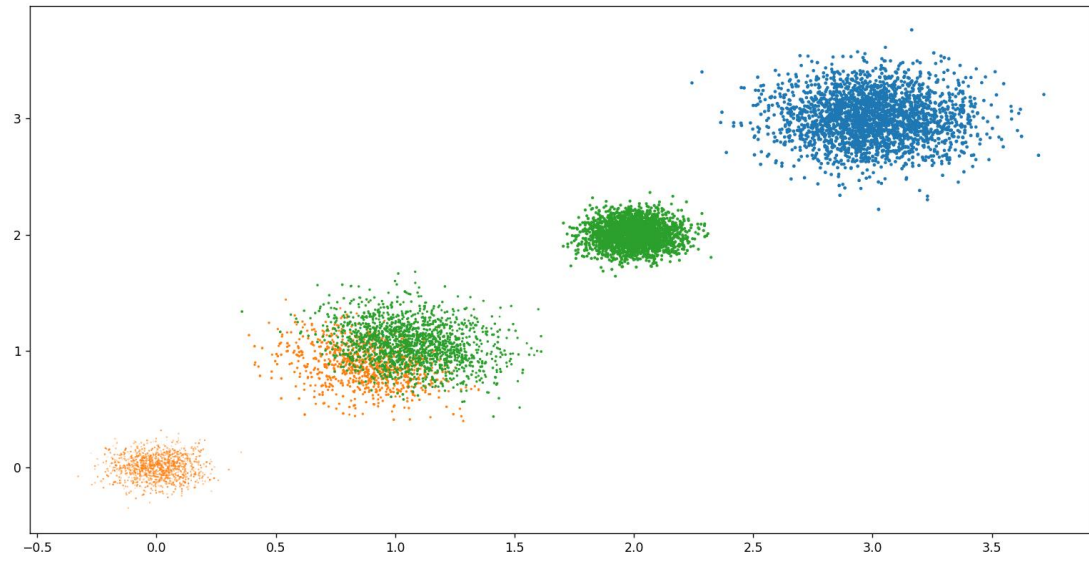
在此步骤中，我们对三个参数进行更改，观察各个参数在模型中的作用：

- 更新 K（簇的数目）：

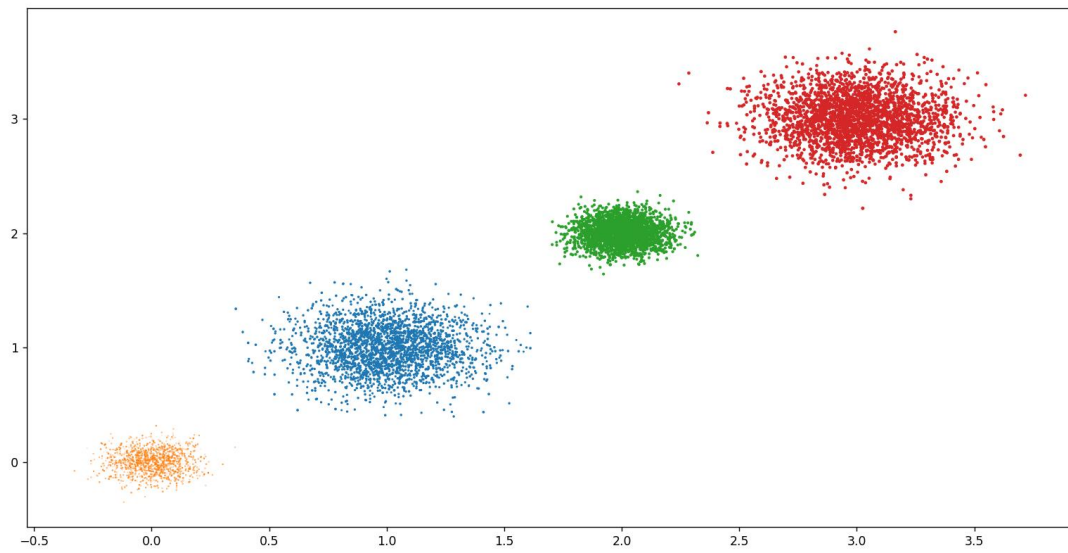
当 $K=2$ ：



当 $K=3$ ：

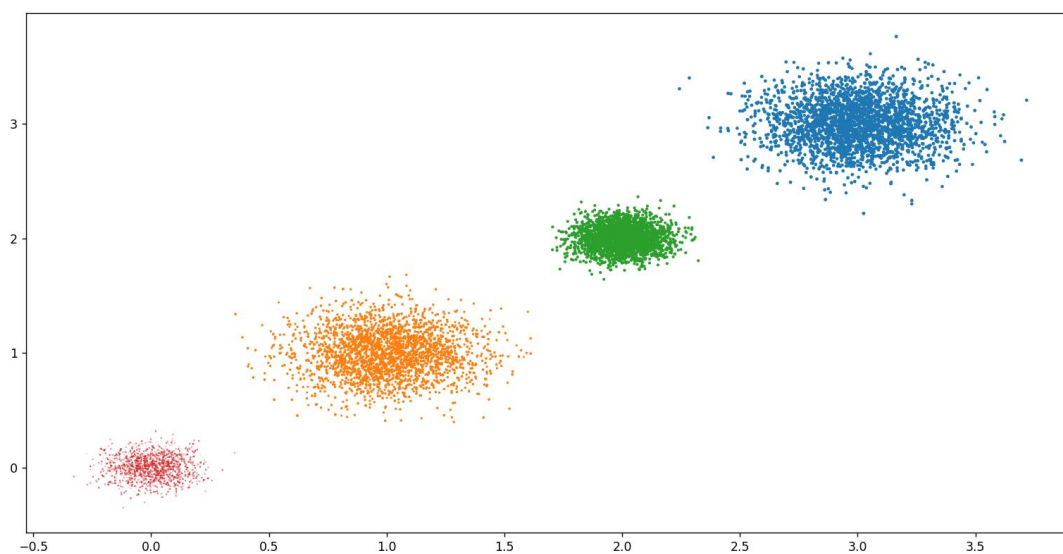


当 $K=4$:

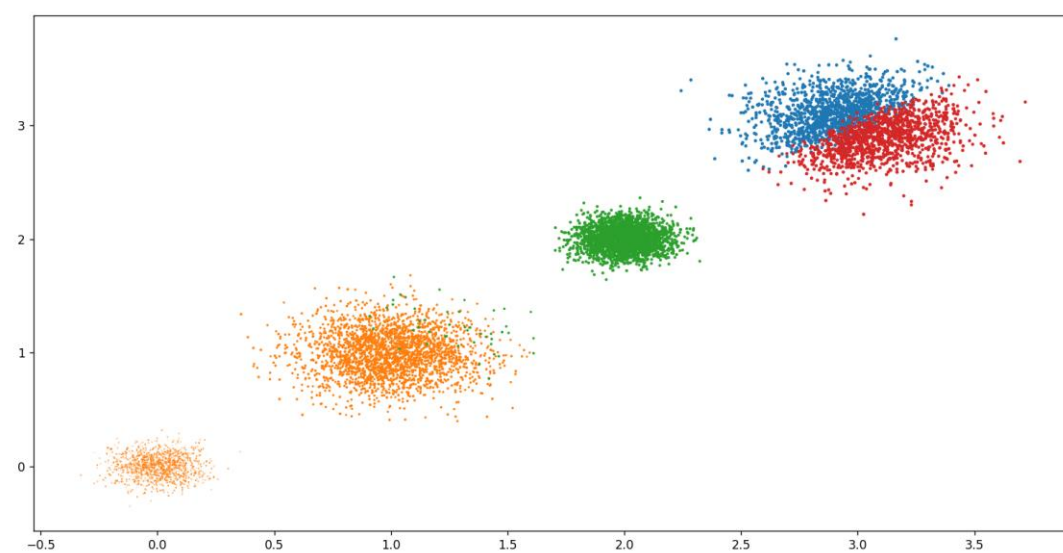


● 更新最大迭代次数:

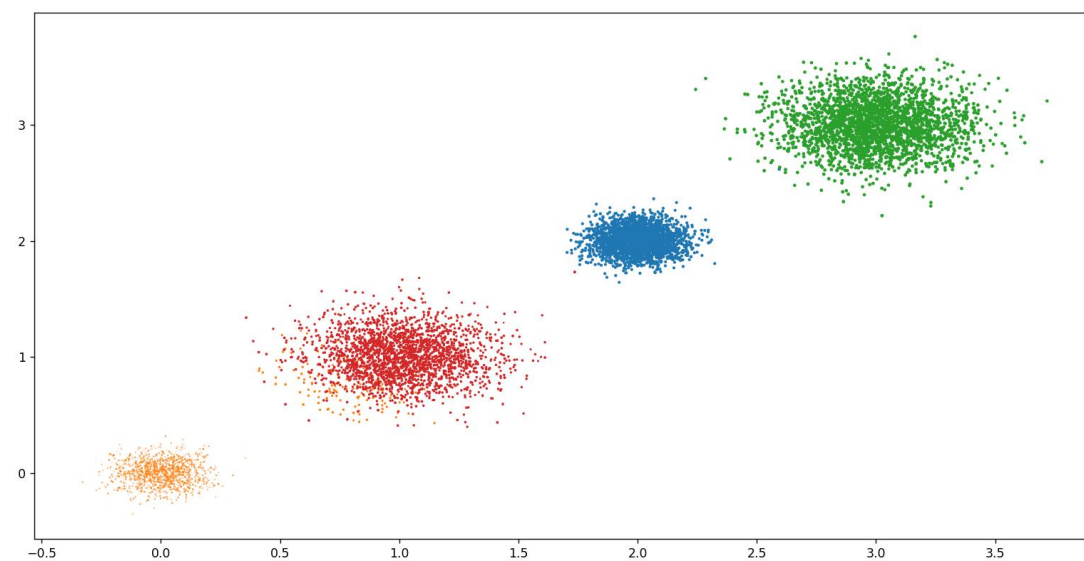
当 $\text{max_iterations}=50$ (效果已经很好):



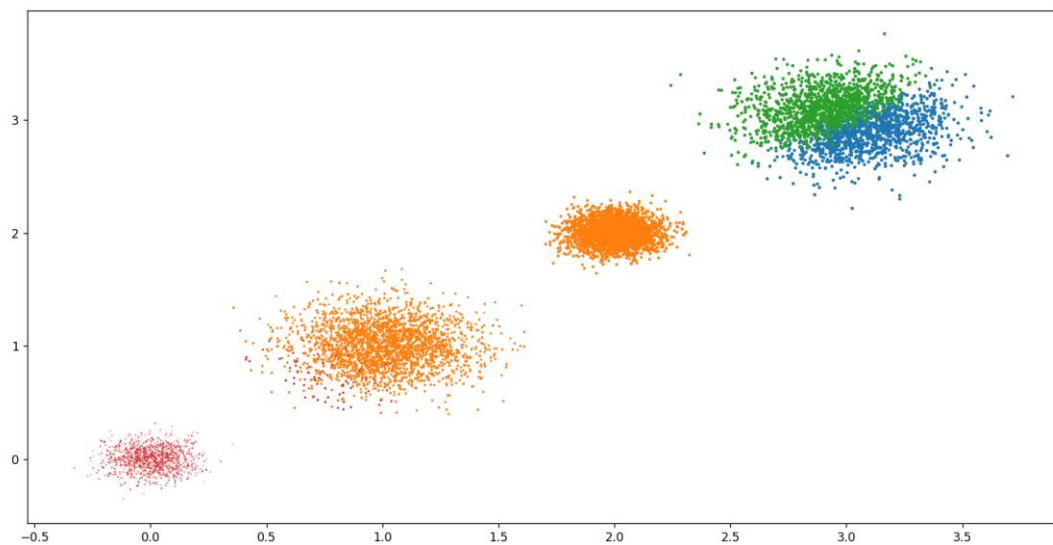
当 $\text{max_iterations}=500$ (效果反而变差):



当 $\text{max_iterations}=5000$, 观察到效果再次变得很好:



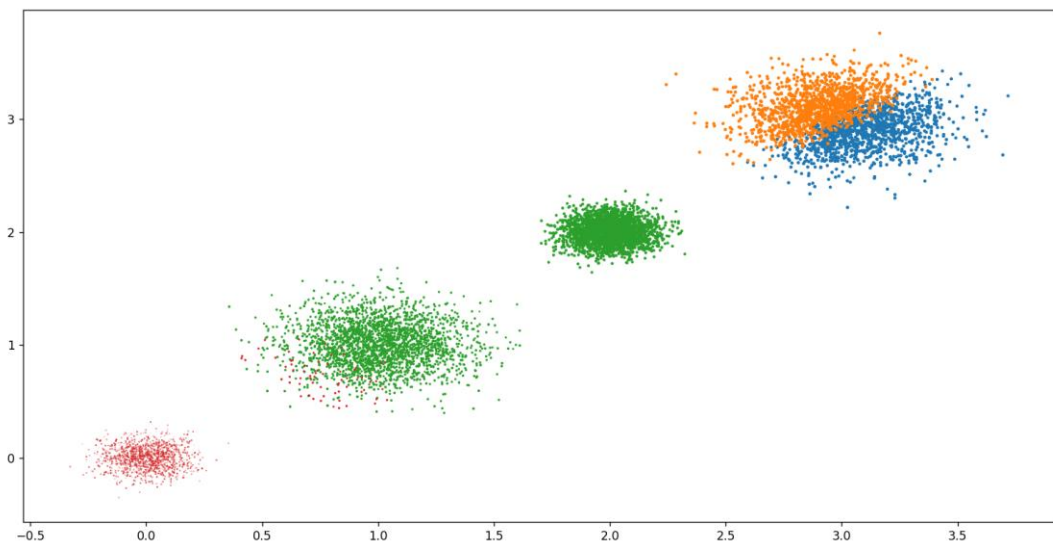
当 $\text{max_iterations}=50000$ (观察到效果再次变差):



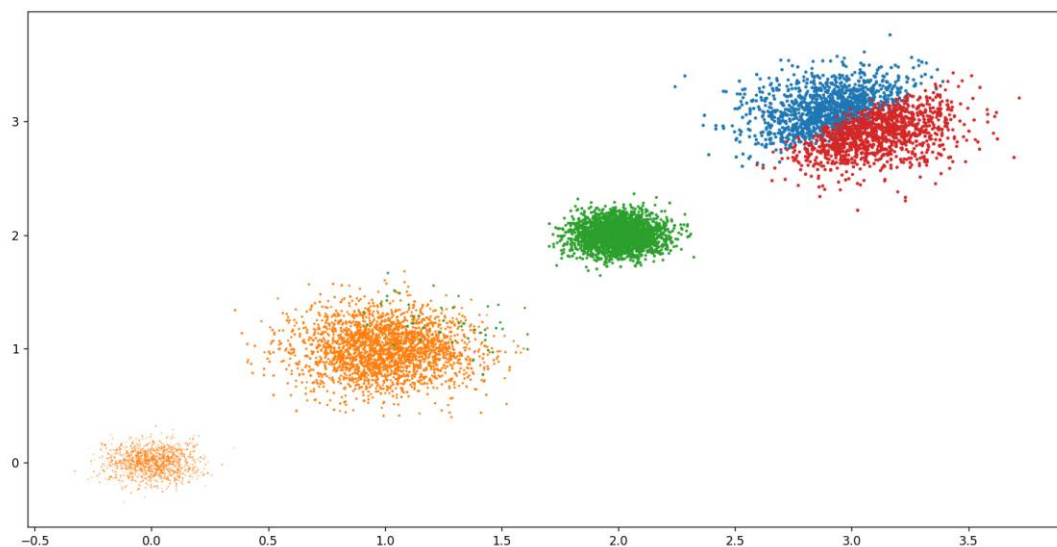
综合这四种情况，我将原因归结为，Kmeans 算法对离群点十分敏感，一旦观察到离群点，整个簇就可能偏移，所以使用该算法时不能追求过高的迭代次数，因为高迭代次数不一定意味着好的效果。

● 更新 varepsilon:

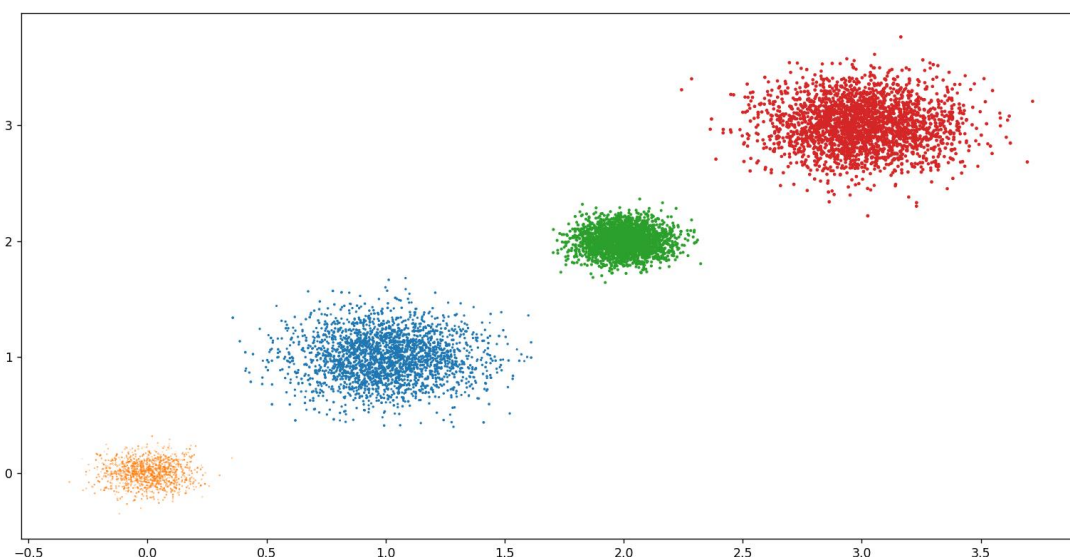
当 $\text{varepsilon}=1e-3$ (效果很差):



当 $\text{varepsilon}=1e-4$ (簇发生了改变，但还是很差):



当 $\text{varepsilon}=1e-5$ (效果终于变好了!) :



影响聚类效果：减小 varepsilon 可以使得算法更容易收敛，因为算法在达到更小的聚类中心变化时就会停止。然而，如果设置过小，可能导致算法过早收敛，得到不够优化的结果。
运行时间：减小 varepsilon 可能会增加算法的运行时间，因为算法需要更小的变化来触发停止条件。

五. 实验分析

我通过对 K-means 聚类算法进行多组实验，调整关键参数 (K、 varepsilon 、 max_iterations)，观察聚类效果和算法运行情况，得到以下结论：

- K-means 对离群点的敏感性：

实验结果再次印证了 K-means 算法对于离群点的敏感性。由于 K-means 基于簇内平方和的最小化，离群点可能对平方和产生较大的影响，导致聚类结果受到离群点的影响。

- K 值的选择：

K 值的选择对聚类效果至关重要。在实验中，通过尝试 K=2、3、4 等不同值，观察聚类效果和 WCSS，可以确定最适合数据集的 K 值。较小的 K 可能导致过于粗糙的聚类，而较大的 K 可能导致细致的划分，需要权衡。

- varepsilon 的影响：

varepsilon 是控制算法收敛的重要参数。较小的 varepsilon 可能使算法更容易收敛，但也可能导致过度拟合。在实验中，观察了不同 varepsilon 值下的收敛情况，选择了适度的 varepsilon 以平衡算法的稳健性和灵敏性。

- max_iterations 的调整：

实验中尝试了不同的 max_iterations 值，观察算法的收敛情况。在一定范围内增加 max_iterations 有助于更好地收敛到最优解，但过大的 max_iterations 可能带来计算开销。需要根据实际情况选择适当的迭代次数。

- 实验应用与参数选择的综合考虑：

在实际应用中，需要根据任务需求、数据特点以及计算资源等因素，综合考虑参数的选择。通过实验，获得了更深入的理解，为在实际场景中应用 K-means 算法提供了指导。

- 迭代观察和调优的重要性：

通过迭代观察聚类效果、收敛情况以及运行时间，能够更全面地了解算法的表现。对参数的调优不仅仅依赖于单一指标，还需要考虑算法的稳健性和实际应用中的可行性。

通过这次实验，我深入了解了 K-means 算法的工作原理和关键参数的影响，为后续在实际问题中应用聚类算法提供了有益的经验 and 启示，大大提高了自己的调参能力和算法实现能力，粗略掌握了了解一个机器学习算法的全流程。

六. 附录

main.py:

```
from sklearn import datasets
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

```

def normalize(X, axis=-1, p=2):
    """对数据集 X 进行正则化。

    Parameters:
    -----
    X : numpy array
        待正则化的数据集。
    axis : int, optional (默认为 -1)
        指定正则化的轴。
    p : int, optional (默认为 2)
        指定正则化的阶数。

    Returns:
    -----
    numpy array
        正则化后的数据集。
    """
    lp_norm = np.atleast_1d(np.linalg.norm(X, p, axis))
    lp_norm[lp_norm == 0] = 1
    return X / np.expand_dims(lp_norm, axis)

def euclidean_distance(one_sample, X):
    """计算一个样本与数据集中所有样本的欧氏距离的平方。

    Parameters:
    -----
    one_sample : numpy array
        待计算欧氏距离的样本。
    X : numpy array
        数据集。

    Returns:
    -----
    numpy array
        包含欧氏距离平方的数组。
    """
    one_sample = one_sample.reshape(1, -1)
    X = X.reshape(X.shape[0], -1)
    distances = np.power(np.tile(one_sample, (X.shape[0], 1)) - X,
2).sum(axis=1)
    return distances

class KMeans:

```

```

"""Kmeans 聚类算法.

Parameters:
-----
k : int, optional (默认为 2)
    聚类的数目.
max_iterations : int, optional (默认为 500)
    最大迭代次数.
varepsilon : float, optional (默认为 0.0001)
    阈值.
    判断是否收敛, 如果上一次的所有 k 个聚类中心与本次的所有 k 个聚类中心的差
都小于 varepsilon,
    则说明算法已经收敛
"""

def __init__(self, k=2, max_iterations=500, varepsilon=0.0001):
    self.k = k
    self.max_iterations = max_iterations
    self.varepsilon = varepsilon

def init_random_centroids(self, X):
    """从所有样本中随机选取 self.k 样本作为初始的聚类中心。

    Parameters:
    -----
    X : numpy array
        数据集.

    Returns:
    -----
    numpy array
        初始的聚类中心.
    """
    n_samples, n_features = np.shape(X)
    centroids = np.zeros((self.k, n_features))
    for i in range(self.k):
        centroid = X[np.random.choice(range(n_samples))]
        centroids[i] = centroid
    return centroids

def closest_centroid(self, sample, centroids):
    """返回距离该样本最近的一个中心索引, 范围[0, self.k)。

    Parameters:
    -----

```

```

sample : numpy array
    待分类的样本.
centroids : numpy array
    当前的聚类中心.

Returns:
-----
int
    距离最近的聚类中心的索引.
"""
distances = euclidean_distance(sample, centroids)
closest_i = np.argmin(distances)
return closest_i

def create_clusters(self, centroids, X):
    """将所有样本进行归类，归类规则就是将该样本归类到与其最近的中心。

    Parameters:
    -----
    centroids : numpy array
        当前的聚类中心.
    X : numpy array
        数据集.

    Returns:
    -----
    list
        包含每个聚类中心对应的样本索引列表的列表.
    """
    n_samples = np.shape(X)[0]
    clusters = [[] for _ in range(self.k)]
    for sample_i, sample in enumerate(X):
        centroid_i = self.closest_centroid(sample, centroids)
        clusters[centroid_i].append(sample_i)
    return clusters

def update_centroids(self, clusters, X):
    """对中心进行更新。

    Parameters:
    -----
    clusters : list
        包含每个聚类中心对应的样本索引列表的列表.
    X : numpy array

```

数据集.

Returns:

numpy array

更新后的聚类中心.

"""

```
n_features = np.shape(X)[1]
centroids = np.zeros((self.k, n_features))
for i, cluster in enumerate(clusters):
    centroid = np.mean(X[cluster], axis=0)
    centroids[i] = centroid
return centroids
```

```
def get_cluster_labels(self, clusters, X):
```

"""将所有样本进行归类，其所在的类别的索引就是其类别标签。

Parameters:

clusters : list

包含每个聚类中心对应的样本索引列表的列表.

X : numpy array

数据集.

Returns:

numpy array

包含每个样本所属类别的标签.

"""

```
y_pred = np.zeros(np.shape(X)[0])
for cluster_i, cluster in enumerate(clusters):
    for sample_i in cluster:
        y_pred[sample_i] = cluster_i
return y_pred
```

```
def predict(self, X):
```

"""对整个数据集 X 进行 Kmeans 聚类，返回其聚类的标签。

Parameters:

X : numpy array

数据集.

Returns:

```

        """
        numpy array
            包含每个样本所属类别的标签。
        """

        centroids = self.init_random_centroids(X)

        for _ in range(self.max_iterations):
            clusters = self.create_clusters(centroids, X)
            former_centroids = centroids

            centroids = self.update_centroids(clusters, X)

            diff = centroids - former_centroids
            if np.all(diff < self.varepsilon):
                break

        return self.get_cluster_labels(clusters, X)

def main():
    # 加载数据集
    X, y = datasets.make_blobs(n_samples=10000,
                               n_features=3,
                               centers=[[0,0,0], [1,1,1], [2,2,2],
[3,3,3]],
                               cluster_std=[0.1, 0.2, 0.1, 0.2],
                               random_state=5)

    # 聚类
    clf = KMeans(k=4, varepsilon=1e-5)
    y_pred = clf.predict(X)

    # 可视化
    fig = plt.figure(figsize=(12, 8))
    ax = Axes3D(fig, rect=[0, 0, 1, 1], elev=30, azimuth=20)
    for i in range(4):
        plt.scatter(X[y_pred==i][:, 0], X[y_pred==i][:, 1],
X[y_pred==i][:, 2])# 画出聚类后得到的数据分布
        # plt.scatter(X[y==i][:, 0], X[y==i][:, 1], X[y==i][:, 2])# 画出
数据最初的分布
    plt.show()

if __name__ == "__main__":
    main()

```